

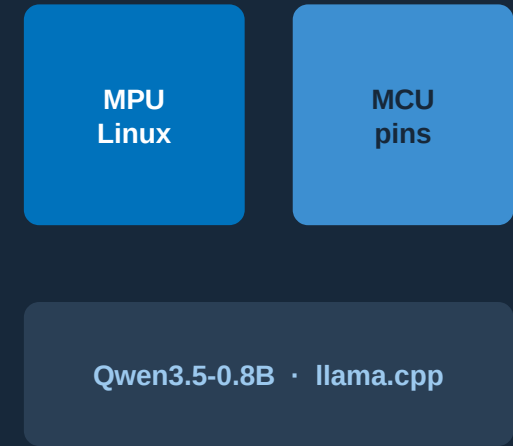
Chat with an AI that lives on an Arduino

A small language model (Qwen3.5-0.8B) runs fully offline on the Arduino UNO Q at the front of the room. Your laptop talks to it over Wi-Fi.

Laptop with Wi-Fi

Any web browser

Python (optional, part 2)



Wi-Fi: GenAI-Lab

So what is a language model, really?

Strip away the hype and it does exactly one thing: guess what comes next.

IT ONLY EVER DOES THIS

“The capital of France is ...”



Paris 91%

Lyon 4%

home 1%

It picks one, adds it to the sentence, and asks the same question again.

1

It reads tokens, not words

Text is chopped into chunks: “un · forget · table”. The model never sees letters — which is why it can explain physics yet miscount the r’s in “strawberry”.

2

Parameters = what it learned

The 0.8B in Qwen3.5-0.8B means 800 million adjustable numbers. They are not a database of facts; they are the settings that make its next-token guesses good.

3

The chat window is its memory

It remembers nothing between sessions. Everything it “knows” about you is the text currently on screen — that window is called the context.

That is the whole trick. Scale it up and you get ChatGPT; shrink it down and it fits on the board in front of you.

Where does the skill come from?

Two very different phases — one costs millions of dollars, the other runs on 3.5 watts.

1

Training

ONCE · IN A DATA CENTRE

- Show the model an enormous amount of text with the last word hidden, and let it guess.
- Every wrong guess nudges those 800 million parameters a tiny bit in a better direction. Repeat trillions of times.
- Then a shorter human-feedback phase teaches it to be helpful and follow instructions rather than just continue text.

2

Inference

EVERY TIME YOU PRESS ENTER

- The parameters are now frozen. Nothing you type changes the model.
- Your prompt goes in, one token comes out, it is appended, and the whole thing runs again — that is why words appear one by one.
- This is the part happening on the Arduino: no training, no internet, just arithmetic on a fixed set of numbers.

INSIDE THE BOX: THE TRANSFORMER

Attention

Each token looks back at every other token and decides which ones matter for what comes next.

Layers

That look-back repeats through dozens of layers, building up meaning from words to sentences.

Probabilities

The final layer scores every possible next token. Sampling picks one — slightly randomly.

Same architecture in every model you have heard of — the board just runs a small one.

Honest limits — read this before you judge it

A next-token guesser has predictable strengths and equally predictable failures.



Plays to its strengths

GIVE IT THESE

Rewriting and reformatting — Summarise, translate, turn messy notes into a table.

Classification — Yes/no calls, sentiment, sorting things into categories.

Explaining and drafting — First drafts and plain-language explanations you will then check.



Fails in these ways

DO NOT TRUST IT HERE

Hallucination — It never says “I don’t know” by default — a fluent wrong answer costs it nothing.

Arithmetic and counting — It predicts what a number looks like; it does not compute.

Fresh or private facts — It only knows what was in its training text. No internet, no news, no you.

RULE OF THUMB

Use it where checking the answer is faster than writing it yourself — and keep a human in the loop.

Now go and find these limits yourself: the board at the front is waiting.

Step 1 — Join the board's Wi-Fi

1

Open your laptop's Wi-Fi menu and join the network below. Your phone works too.

2

Ignore the “no internet” warning — that is the point. Everything runs on the board.

3

Open a browser and go to the address on the right. You should see a chat page.

NETWORK

GenAI-Lab

PASSWORD

edgeai2026

THEN OPEN

`http://10.42.0.1:8081`

If the page will not load: forget other Wi-Fi networks, confirm you are on GenAI-Lab, and retype the address exactly — it is http, not https.

Step 2 — Explore the chat interface

The page at 10.42.0.1:8081 is llama.cpp's built-in GUI — a mini ChatGPT served by the Arduino.

1

Chat

Ask short, concrete questions. The model answers in real time — you will see tokens appear one by one (~5 per second).

2

Vision

Click the attach icon to upload a photo and ask about it. Keep images under 640×640 pixels; expect about a minute per image.

3

Take turns

One board, one request at a time. If it feels stuck, a classmate's request is probably running — watch the queue.

4

Good tasks

Classification, extraction, short explanations. Example: "Temp 30 °C, humidity 85%, standing water — mosquito risk? Answer yes/no + one reason."

What is actually running? (spoiler: not the cloud)

Arduino UNO Q — a dual-brain board

MPU — the thinker

Qualcomm QRB2210
4× A53 cores, Debian Linux
runs llama.cpp + the model

MCU — the doer

STM32U585
real-time pins, LEDs, sensors
classic Arduino sketches

The model: Qwen3.5-0.8B (8-bit) — 0.8 billion parameters, ~1.1 GB of RAM, understands text AND images, 201 languages, fully offline.

“Is this Ollama?” — close cousin.

Ollama is the popular tool for running local LLMs on laptops and Raspberry Pi — easy, but too heavy for this board.

llama.cpp is the lightweight engine underneath that world. It fits the UNO Q and serves the same OpenAI-style API — so everything you learn here transfers directly to Ollama and to cloud APIs.

3.5 W

peak power — no fan

~5 tok/s

generation speed

100%

offline & private

Same recipe scales up: your phone's AI assistant and data-center LLMs work exactly this way — just bigger.

Step 3 — Use it from Python

The board speaks the standard OpenAI API. Stay on GenAI-Lab Wi-Fi and run:

```
# pip install openai
from openai import OpenAI

client = OpenAI(
    base_url="http://10.42.0.1:8081/v1",
    api_key="none") # any string works

resp = client.chat.completions.create(
    model="qwen",
    messages=[{"role": "user", "content":
        "Mosquito risk at 30C, 85% humidity?"}],
    max_tokens=150)

print(resp.choices[0].message.content)
```

»

Same code, three worlds

Swap `base_url` and this exact script talks to Ollama on your laptop or to cloud APIs.

»

Vision too

Send a base64 image as an `image_url` content part — the standard OpenAI vision format.

»

Share the board

Requests queue one by one. Keep `max_tokens` small in class.

Your turn — three challenges

1

Prompt engineer

In the web GUI, get the model to answer a yes/no classification with exactly one sentence of reasoning. How reliable is it over 5 tries?

2

Vision test

Upload a photo of something on your desk. Where does the model do well — and where does it hallucinate details?

3

Python benchmark

From Python, time 3 requests and estimate tokens/second. Compare with the ~5 tok/s we promised.

Remember: the whole thing runs on 3.5 watts with no internet. Small models, honest limits — keep a human in the loop.

Wi-Fi GenAI-Lab · password edgeai2026 · <http://10.42.0.1:8081>